Supplementary Document
Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

This document carries what the main paper's page budget could not. Nothing here introduces a claim the paper does not already make: each item is the tabular or graphical form of a number the Results section gives, or a measurement the paper cites without room to print. Assignment ids (R1–R5, C1–C6), the arm names, and every metric definition are the main paper's. Values are generated from `results/` by the released scripts (`run.py tables`, `run.py figures`, `run.py labelagree`); none is transcribed.

What Escalation Costs, Assignment by Assignment

Table S1 gives the escalation rate e per assignment, the calls it buys, the local search time, and the one-off build. The rate is what the cost argument turns on: an assignment that escalates rarely is one whose frozen tool is carrying the class on its own, and the two assignments above 20% are the two where the reference pool is thinnest relative to the variety of the submissions.

	e (%)	calls	$\mathrm{ATT}_{\mathrm{cpu}}$ (s)	build (min)
R1	0.7	8	2.28	32
R2	22.5	196	3.18	27
R3	0.0	0	1.83	18
R4	9.2	66	3.43	28
R5	0.9	2	4.79	18
C1	13.2	116	3.17	20
C2	23.3	261	2.22	38
C3	13.0	18	6.94	3
C4	5.3	4	1.34	12
C5	15.6	24	4.86	4
C6	66.7	4	6.58	6
All	**11.8**	**699**	**2.88**	**206**

Table S1: Escalation and cost per assignment, `claude-sonnet-5`.

Robustness

Table S2 puts three axes side by side: assignment difficulty, the choice of builder, and the escalation threshold. The difficulty bands group the eleven assignments as CS1, Codeforces 800–1000, and Codeforces 1200–2000.

Stratum	Refactory	PyDex RR (%) / RPS	$\mathrm{CoEP}_{\mathrm{LF}}$
Difficulty (`claude-sonnet-5`)			
CS1	90.8 / 0.397	98.6 / 0.382	99.4 / 0.279
CF 800–1000	83.1 / 0.663	99.4 / 0.942	100.0 / 0.572
CF 1200–2000	54.5 / 0.815	100.0 / 0.800	100.0 / 0.693
Builder			
`claude-sonnet-5`	–	99.0 / 0.598	99.7 / 0.404
`gpt-5.4-mini`	–	98.9 / 0.669	99.7 / 0.417
Threshold, 29 measured settings			
range	–	–	99.7 / 0.378–0.451
spend		\$12.47–\$15.61 total	

Table S2: Difficulty, builder, and escalation threshold.

Figure S1 plots every measured threshold setting against the baselines it is compared to. The point is the emptiness between the cluster and the baselines: the whole family spans \$12.47–\$15.61 of total spend, so which setting ships cannot change a conclusion. The shipped rule, escalate a passing local patch whose RPS exceeds 1, is marked with a star and is the canonical run rather than the sweep's reconstruction of it.

Do the Rule Names Say What the Rules Do?

The paper claims an auditable name for every repair its rules make, and this is the measurement behind that claim. Three raters independently judged all **195** rules the two builders wrote, each against the rule's own source and one repair it actually produced, preferring a repair no other rule contributed to so the edit is attributable. Four rules the rating guide had worked through as examples are excluded, leaving **191**.

The scheme has three values, applied in a fixed order so that the two questions do not blur. A name is `wrong` if it contradicts the rule's main behaviour; otherwise `accurate` if it identifies a misconception, and `describes_edit` if it states the edit correctly without saying what the student misunderstood. Only the first of these bears on the paper's claim; the second distinction is reported here for completeness.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

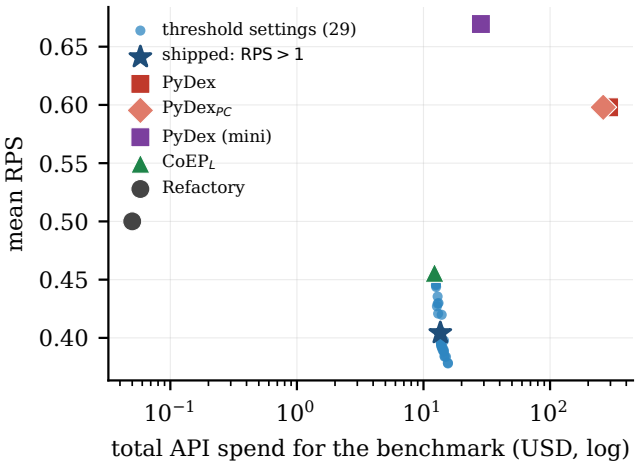


Figure S1: Mean RPS against total spend. Each small marker is one threshold setting; the star is the shipped rule. Repair rate is identical across all of them.

	accurate	describes_edit	wrong
Rater 1	46.6	45.5	7.9
Rater 2	47.6	42.9	9.4
Rater 3	50.3	42.4	7.3
Majority, per rule	47.1	44.5	**8.4**
Majority, per repair	37.3	62.0	**0.7**

Table S3: Rule-name labelling, in per cent. Per-rule rows count the 191 rules; the per-repair row weights each rule by the repairs it produced, which is the side a student sees.

Agreement is Fleiss' $\kappa = 0.890$ over three raters and three categories, with all three agreeing on 173 of the 191 rules (90.6%); pairwise raw agreement is 94.8%, 93.2% and 93.2%. The per-rater marginals in Table S3 are close enough that no rater is driving the result.

Two asymmetries are worth stating. All **16** names judged `wrong` were written by `gpt-5.4-mini`; `claude-sonnet-5` wrote none, and **14** of the 16 never fire, which is why a rate of 8.4% of rules is 0.7% of repairs. And 66 of the 195 rules never produced a repair, so the per-rule and per-repair rows answer different questions: the first is what the builder wrote, the second is what a class would actually see.

What the Builder Writes

Table S4 lists the rules that carry the most submissions, counted as distinct submissions rather than as applications, since a repair may compose several rules. The distinction that matters is reuse: of the 128 rules the builders wrote and that fired, exactly one fired on more than one assignment, `add_missing_call_parens`, which describes a Python trap rather than anything about a problem. The always-on rules recur everywhere by construction, which is why the builder's layer is priced per assignment and they are not.

Rule	where	submissions	assignments
`generic_flip_compare`	always-on	263	10
`append_missing_return_len_after_loop`	R1	114	1
`slice_case_check_to_equality`	C2	108	1
`generic_shift_index`	always-on	74	5
`strict_one_should_be_exactly_one`	R2	70	1
`count_compare_relax`	R2	58	1
`generic_shift_range`	always-on	50	7
`capitalize_title_to_swapcase`	C2	43	1
128 builder-written rules fired, 1 on more than one assignment			

Table S4: The eight rules repairing the most submissions.

Self-Learning

The eight rules self-learning absorbed read as misconceptions in the same way. On R2 with `claude-sonnet-5`: `counter_var_confusion`, `uniqueness_count_gt1_to_neq1`, `index_vs_count_and_uncalled_bool`, `missing_else_false`. On R2 with `gpt-5.4-mini`: `count_equality_for_uniqueness`, `count_uniqueness_instead_of_direct_membership`, `missing_call_and_predicate_fix`. On C3: `binarysearch_zero_exit`.

Reproducibility

The repository contains the harness, every prompt, the 22 synthesized repairers with their build costs, the per-submission event log each arm wrote, and the rating sheet with all three raters' returned labels. Five commands regenerate every number in both documents from those logs without touching a model: `run.py tables`, `run.py figures`, `run.py numbers`, which recomputes each quantity and reports whether the string still occurs in the paper, `run.py stats`, and `run.py labelagree`. The measured threshold settings are archived as `results/threshold_sweep.json`, and the labelling sheet, its guide and the three returned files as `results/rules*.csv` and `results/labeling_guide.md`.

Two dependencies are outside our control and worth stating. Both builders are closed commodity models reached through their providers' APIs, so a rebuild cannot be guaranteed to reproduce the same rules; what is released is the tools those builds produced, which are ordinary Python and run offline. Refactory's replication package is its authors' and is cloned rather than vendored.